

Three fixes – alignment, false green rows, changed fields only

Part A is one line. Part B is a real bug: a row turns green and locks even when the save was rejected. Part C changes what a deviation stores, following Benoit's answer that only the fields which differ from the configuration are saved.

Part	Problem	Steps
A	Manage Filtering Laws sits on the left of the button row	Step 1
B	Alarm Data Error is shown, nothing is saved, but the row is green and can no longer be edited	Step 2
C	A deviation stores all thirteen parameters even when only one was changed	Steps 3 to 7

Part A – button alignment

Step 1 — HealthAlarmsPreferencePage → contributeButtons

mgs-health-app/.../preferences/healthalarms/HealthAlarmsPreferencePage.java

Add one line to the method you already have. Without layout data the button takes the left of its cell; SWT.END pushes it to the right, beside Apply.

```
@Override
protected void contributeButtons(Composite parent) {
    ((GridLayout) parent.getLayout()).numColumns++;
    Button manageLaws = new Button(parent, SWT.PUSH);
    manageLaws.setText(Messages.CORE.getString("filteringLawManageButton"));
    manageLaws.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false)); // <<< add
    manageLaws.addListener(SWT.Selection, e -> hac.openFilteringLawDialog());
}
```

GridData is already imported in that class.

Part B – a rejected row must not stay green

What happens today: the BCR-6605 block marks the row deviated as soon as it differs from the baseline, and only then do the existing checks run. When one of them fails, isValidData returns false with the flag already set. The row is painted green by the label provider and locked by the cell modifier, yet nothing reached the database.

In the screenshot the MOD35 rows are RED at Low and AMBER at High, which is exactly what the red-against-amber comparison rejects.

The fix is to mark nothing until every check has passed. Rather than clearing the flag at each of the four return false points, the block moves to the end of the method.

Step 2 — HealthAlarmsPreferenceStore → isValidData

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

Two edits in the same method.

2.1 Delete the BCR-6605 block from inside the threshold loop. It sits just before if (currentModel.getSeverity() == Severity.RED).

```
// DELETE all of this
if (currentModel.isModifiedAgainstBaseline()) {
    currentModel.setDeviated(true);
}
if (currentModel.isDeviated() && (currentModel.getTeReference() == null
    || currentModel.getTeReference().trim().isEmpty())) {
    MessageDialog.openError(...);
    currentModel.setDeviated(false);
    return false;
}
```

2.2 Add this immediately before this.data = dataToBeValidated; at the end of the method.

```

// BCR-6605 every check above has passed, so now decide which rows are deviations.
// Collect first, validate second, mark last: that way a missing TE on the tenth row
// cannot leave the first nine green.
List<WCThresholdModel> deviatedModels = new ArrayList<>();
for (Entry<Aircraft, WCMechanicalComponentModel> entry : dataToBeValidated.entrySet()) {
    for (WCThresholdModel model : getComponentThresholds(entry.getValue()).values()) {
        if (model.isDeviated() || model.isModifiedAgainstBaseline()) {
            deviatedModels.add(model);
        }
    }
}

for (WCThresholdModel model : deviatedModels) {
    if (model.getTeReference() == null || model.getTeReference().trim().isEmpty()) {
        MessageDialog.openError(parent.getShell(),
            Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
            MessageFormat.format(Messages.CORE.getString("deviationErrorTeMandatory"),
                model.getIndicator().getName()));
        return false;
    }
}

for (WCThresholdModel model : deviatedModels) {
    model.setDeviated(true);
}

```

A row already loaded as a deviation is in the list too, but it always has a TE, so it passes and is simply marked again. Nothing is created twice – saveDeviations still skips any row that already has a deviationId.

Part C – store only the fields that changed

Benoit's answer: a deviation records what differs from the configuration, not a full copy of the threshold. Severity, TE reference and filtering law are always stored; every other parameter is written only when the user actually changed it, and left null otherwise.

Always stored	Stored only when changed
Severity	Threshold level, type, value
TE reference	LT min, LT max, learning time
Filtering law	Work card, LT min deviation, LT mean factor
Start date	LT deviation coefficient, successive points

This needs the baseline to hold every parameter, not just the three it holds today. That is Step 3, and it improves change detection as a side effect: editing the work card or the successive points now counts as a deviation, which it did not before.

Consequence for Part D. A deviation is now a patch rather than a snapshot, so when the analysis applies one it must read each null field from the configuration. Worth noting in the tracker before Part D starts.

Step 3 — WCThresholdBaseline – hold every parameter

`com.eurocopter.mgs.business.health.model.WCThresholdBaseline`

Replace the whole class. The copy constructor takes the model at the moment it still holds configuration values, which keeps the call site to one line.

```

package com.eurocopter.mgs.business.health.model;

import java.io.Serializable;

import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdLevel;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdType;

/**
 * BCR-6605 the configuration values of one threshold, captured before any deviation is
 * applied on top. Used to detect what the user changed and to revert on acknowledge.
 */
public class WThresholdBaseline implements Serializable {

    private static final long serialVersionUID = 1L;

    private final ThresholdLevel level;
    private final ThresholdType type;
    private final Double defaultValue;
    private final Double minValue;
    private final Double maxValue;
    private final Long learningTime;
    private final String workCardReference;
    private final Double minimumDeviation;
    private final Double meanFactor;
    private final Double deviationCoefficient;
    private final Integer nbSuccessivePoints;
    private final String filteringLawName;

    public WThresholdBaseline(WThresholdModel model, String filteringLawName) {
        this.level = model.getLevel();
        this.type = model.getType();
        this.defaultValue = model.getDefaultValue();
        this.minValue = model.getMinValue();
        this.maxValue = model.getMaxValue();
        this.learningTime = model.getLearningTime();
        this.workCardReference = model.getWorkCardReference();
        this.minimumDeviation = model.getMinimumDeviation();
        this.meanFactor = model.getMeanFactor();
        this.deviationCoefficient = model.getDeviationCoefficient();
        this.nbSuccessivePoints = model.getNbSuccessivePoints();
        this.filteringLawName = filteringLawName;
    }

    public ThresholdLevel getLevel() { return level; }
    public ThresholdType getType() { return type; }
    public Double getDefaultValue() { return defaultValue; }
    public Double getMinValue() { return minValue; }
    public Double getMaxValue() { return maxValue; }
    public Long getLearningTime() { return learningTime; }
    public String getWorkCardReference() { return workCardReference; }
    public Double getMinimumDeviation() { return minimumDeviation; }
    public Double getMeanFactor() { return meanFactor; }
    public Double getDeviationCoefficient() { return deviationCoefficient; }
    public Integer getNbSuccessivePoints() { return nbSuccessivePoints; }
    public String getFilteringLawName() { return filteringLawName; }
}

```

Check the getter names against your `WThresholdModel` before pasting – these are the ones `toDeviationDto` already calls, so they should match, but `getWorkCardReference` and `getMinimumDeviation` are the two most likely to be spelled differently.

Step 4 — `WThresholdModel` → `isModifiedAgainstBaseline`

Replace the method. It compared three fields; now it compares all twelve.

```

public boolean isModifiedAgainstBaseline() {
    if (baseline == null) {
        return false;
    }
    return !Objects.equals(baseline.getLevel(), getLevel())
        || !Objects.equals(baseline.getType(), getType())
        || !Objects.equals(baseline.getDefaultValue(), getDefaultValue())
        || !Objects.equals(baseline.getMinValue(), getMinValue())
        || !Objects.equals(baseline.getMaxValue(), getMaxValue())
        || !Objects.equals(baseline.getLearningTime(), getLearningTime())
        || !Objects.equals(baseline.getWorkCardReference(), getWorkCardReference())
        || !Objects.equals(baseline.getMinimumDeviation(), getMinimumDeviation())
        || !Objects.equals(baseline.getMeanFactor(), getMeanFactor())
        || !Objects.equals(baseline.getDeviationCoefficient(), getDeviationCoefficient())
        || !Objects.equals(baseline.getNbSuccessivePoints(), getNbSuccessivePoints())
        || !Objects.equals(baseline.getFilteringLawName(), getFilteringLawName());
}

```

Step 5 — HealthAlarmsPreferenceStore → getThresholdModels – two changes

Both inside the loop that sets the law and the baseline.

5.1 The baseline now takes the model itself. Replace the one line:

```

// BEFORE
thresholdModel.setBaseline(new WCThresholdBaseline(
    thresholdModel.getDefaultValue(), thresholdModel.getMinValue(), law));

// AFTER
thresholdModel.setBaseline(new WCThresholdBaseline(thresholdModel, law));

```

5.2 The overlay must skip null fields, because a deviation no longer carries all of them. Replace the whole block inside if (deviation != null):

```

if (deviation != null) {
    thresholdModel.setDeviated(true);
    thresholdModel.setDeviationId(deviation.getId());
    thresholdModel.setTeReference(deviation.getTeReference());
    thresholdModel.setDeviationStartDate(deviation.getStartDate());
    thresholdModel.setDeviationEndDate(deviation.getEndDate());

    // BCR-6605 a deviation stores only what changed, so null means "keep the configuration"
    if (deviation.getThresholdLevel() != null) {
        thresholdModel.setLevel(ThresholdLevel.valueOf(deviation.getThresholdLevel()));
    }
    if (deviation.getThresholdType() != null) {
        thresholdModel.setType(ThresholdType.valueOf(deviation.getThresholdType()));
    }
    if (deviation.getThresholdValue() != null) {
        thresholdModel.setDefaultValue(deviation.getThresholdValue());
    }
    if (deviation.getMinValue() != null) {
        thresholdModel.setMinValue(deviation.getMinValue());
    }
    if (deviation.getMaxValue() != null) {
        thresholdModel.setMaxValue(deviation.getMaxValue());
    }
    if (deviation.getLearningTime() != null) {
        thresholdModel.setLearningTime(deviation.getLearningTime());
    }
    if (deviation.getWorkcard() != null) {
        thresholdModel.setWorkCardReference(deviation.getWorkcard());
    }
    if (deviation.getMinimumDeviation() != null) {
        thresholdModel.setMinimumDeviation(deviation.getMinimumDeviation());
    }
    if (deviation.getMeanFactor() != null) {
        thresholdModel.setMeanFactor(deviation.getMeanFactor());
    }
    if (deviation.getDeviationCoefficient() != null) {
        thresholdModel.setDeviationCoefficient(deviation.getDeviationCoefficient());
    }
    if (deviation.getNbSuccessivePoints() != null) {
        thresholdModel.setNbSuccessivePoints(deviation.getNbSuccessivePoints());
    }
    if (deviation.getFilteringLawName() != null) {
        thresholdModel.setFilteringLawName(deviation.getFilteringLawName());
        thresholdModel.setFilteringLawId(deviation.getFilteringLawId());
    }
}
}

```

Without these guards a null level would reach `ThresholdLevel.valueOf(null)` and throw on every screen load, so this guard is not optional once Step 6 starts writing nulls.

Step 6 — HealthAlarmsPreferenceStore → toDeviationDto

Replace the whole method. Severity, TE and the filtering law are always written; the rest only when they differ from the baseline.

```

private IndicatorDeviationDto toDeviationDto(WCThresholdModel model, Date now) {
    IndicatorDeviationDto dto = new IndicatorDeviationDto();
    dto.setIndicatorConfigurationId(model.getIndicatorConfigurationId());
    dto.setStartDate(now);

    // BCR-6605 always stored
    dto.setSeverity(model.getSeverity().name());
    dto.setTeReference(model.getTeReference());
    dto.setFilteringLawId(getFilteringLawId(model.getFilteringLawName()));

    // BCR-6605 stored only when the user changed them
    WCThresholdBaseline baseline = model.getBaseline();
    if (baseline == null) {
        return dto;
    }
    if (!Objects.equals(baseline.getLevel(), model.getLevel())) {
        dto.setThresholdLevel(model.getLevel().name());
    }
    if (!Objects.equals(baseline.getType(), model.getType())) {
        dto.setThresholdType(model.getType().name());
    }
    if (!Objects.equals(baseline.getDefaultValue(), model.getDefaultValue())) {
        dto.setThresholdValue(model.getDefaultValue());
    }
    if (!Objects.equals(baseline.getMinValue(), model.getMinValue())) {
        dto.setMinValue(model.getMinValue());
    }
    if (!Objects.equals(baseline.getMaxValue(), model.getMaxValue())) {
        dto.setMaxValue(model.getMaxValue());
    }
    if (!Objects.equals(baseline.getLearningTime(), model.getLearningTime())) {
        dto.setLearningTime(model.getLearningTime());
    }
    if (!Objects.equals(baseline.getWorkCardReference(), model.getWorkCardReference())) {
        dto.setWorkcard(model.getWorkCardReference());
    }
    if (!Objects.equals(baseline.getMinimumDeviation(), model.getMinimumDeviation())) {
        dto.setMinimumDeviation(model.getMinimumDeviation());
    }
    if (!Objects.equals(baseline.getMeanFactor(), model.getMeanFactor())) {
        dto.setMeanFactor(model.getMeanFactor());
    }
    if (!Objects.equals(baseline.getDeviationCoefficient(), model.getDeviationCoefficient())) {
        dto.setDeviationCoefficient(model.getDeviationCoefficient());
    }
    if (!Objects.equals(baseline.getNbSuccessivePoints(), model.getNbSuccessivePoints())) {
        dto.setNbSuccessivePoints(model.getNbSuccessivePoints());
    }
    return dto;
}

```

Import `java.util.Objects` if it is not already there.

The filtering law is written every time, even unchanged, because Benoit asked for it always to be present. That also means the Deviation List can show which law was in force without going back to the configuration.

Step 7 — DeviationTableComposite → getColumnText – handle the nulls

The table now receives rows with empty parameters. Three columns return a String straight from the DTO and would show the word null, or throw. Replace those three cases.

```

case 1: return d.getSeverity() != null ? d.getSeverity() : "";
case 2: return d.getThresholdLevel() != null ? d.getThresholdLevel() : "";
case 3: return d.getThresholdType() != null ? d.getThresholdType() : "";

```

The number columns already go through `number()`, which returns an empty string for null, and the law columns are already guarded.

This is also REQ 6 arriving for free: the Deviation List now shows only the parameters the user modified, because everything else is blank. The note in the tracker about REQ 6 being deferred can be closed.

Build and test

#	Action
1	Rebuild the client only. No server or database change in this set.

#	Action
2	Part A – open Health: Manage Filtering Laws now sits on the right, next to Apply
3	Part B – edit MOD35 so RED is Low and AMBER is High, add a TE, press Apply. The Alarm Data Error appears, and the row stays its normal colour and is still editable
4	Fix the levels, press Apply again, OK – now the row turns green
5	Part C – change only the Value on one row, add a TE, Apply, OK
6	SELECT * FROM public.indicator_deviation – expect severity, te_reference, filtering_law_id, start_date and threshold_value filled, and every other parameter column null
7	Open the Deviation list tab – only the changed value shows, the rest of the row is blank
8	Reopen the Health page – the row is green and shows the deviated value, with all its other parameters still at their configuration values

Item 8 is the one that proves the null guards in Step 5.2 work. If the screen fails to load with a NullPointerException or an IllegalArgumentException from valueOf, one of the guards is missing.